



Database schema and structure versioning

Overview

Keeping track of your application's database is not an easy task. Database schemas tend to mismatch in different environments, data in one of the databases may miss some crucial piece of data. Such occasions can be irritating, especially when caught in production.

The situation gets worse when you develop redistributable software. In this case, each of your clients has their own database instance whose structure may differ from others'. In such a project, keeping track of your clients' databases can become a nightmare.

Let's look at the database versioning best practices that help us deal with this problem.

1. Database versioning: the problem

When you are a single programmer working on a project that is not yet shipped to production, there is no such problem as database versioning. You just change your DB schema the way you want and it always works.

Problems arise when your software starts operating in production or a new team member joins you to work on database-related parts of your project. As soon as you have more than one database instance, they start getting out of sync. Even with a single instance, it takes a significant amount of time to synchronize the changes when more than one developer work with it.

2. Database versioning best practices

Fortunately, we are not alone. There are plenty of materials written on that topic as well as software that is aimed to solve this problem. It is an ultimate guideline for how to evolve your database along with the code that uses it.

Alright, so what are these database versioning best practices?

Best practice #1: we need to treat the application database and the reference data in it as regular code. That means we should store both its schema and the reference data in a source control system.

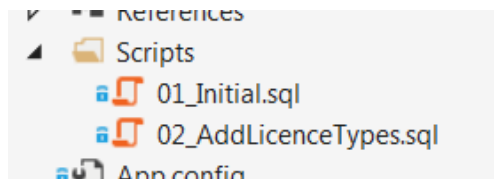
Note that this rule includes not only schema of the database but also the reference data in it. Reference data is the data that is mandatory to run the application. For example, if you have a dictionary of all customer types possible on which existence your application relies, you should store it in the source control system as well.

2. Database versioning best practices

Best practice #2: we have to store every change in the database schema and in the reference data explicitly. This means that for every modification we make we should create a separate SQL script with the changes. If the modification affects both the schema and the reference data, they should be reflected in a single script.

Adhering to this rule is a vital part of building a successful database versioning system. Many projects have their database schema stored in a source control but often it is just a snapshot of the latest database version whatever that is. All the changes in it are tracked by the source control system itself, they are not stored explicitly. Moreover, often the changes in the reference data are not tracked at all.

Such tools as Visual Studio database project emphasize that approach and urge programmers to use auto-generated upgrade scripts for schema update. While this may work well in small projects, in larger projects, tracking changes in the database using auto-generated scripts becomes a burden.



2. Database versioning best practices

Best practice #3: every SQL script file must be immutable after it is deployed to production or staging environment.

The whole point of storing the changes in separate files is to be able to track each of them. When we modify the existing SQL scripts we lose all the benefits the database versioning best practices provide us. Keep the script files unchangeable after their deployment. If you need to turn down the changes that are already shipped - create a separate script for that.

Best practice #4: all changes in the database's schema and reference data have to be applied through the scripts. Neither of them can be applied manually.

If we modify the database passing over our scripts, the whole idea of database versioning becomes worthless, so we need to make sure the changes are made only via the SQL scripts we create.

2. Database versioning best practices

Best practice #5: every developer in the team should have their own database instance.

Often, teams start with a single database in the developer environment. That works well at the beginning but when the database grows large enough, simultaneous modifications of it become harder and harder until at some point stop working at all.

Changes programmers make are often incompatible so it's a good idea for each programmer to have a separate DB instance to avoid such collisions. If developers do modify related pieces of the DB schema simultaneously, such conflicts can be resolved using a source control system, just like the conflicts in C#/Java/etc code.

Moreover, if you have several branches of your code base, you might also want to create a separate DB instance for each of them, depending on how different the databases in these branches are.

2. Database versioning best practices

Best practice #6: database version should be stored in the database itself. I usually tend to create a separate table named Settings and keep the version there. Don't use complex notations like "x.y.z" for the version number, just use a single integer.

3. What are the benefits of such approach?

So what benefits these database versioning best practices give us?

The first and the most important advantage is that when we use this approach, we don't have the problems with the database schema mismatch anymore. Automatic upgrades to the latest version solve them completely, of course if we fully adhere to the rules described above.

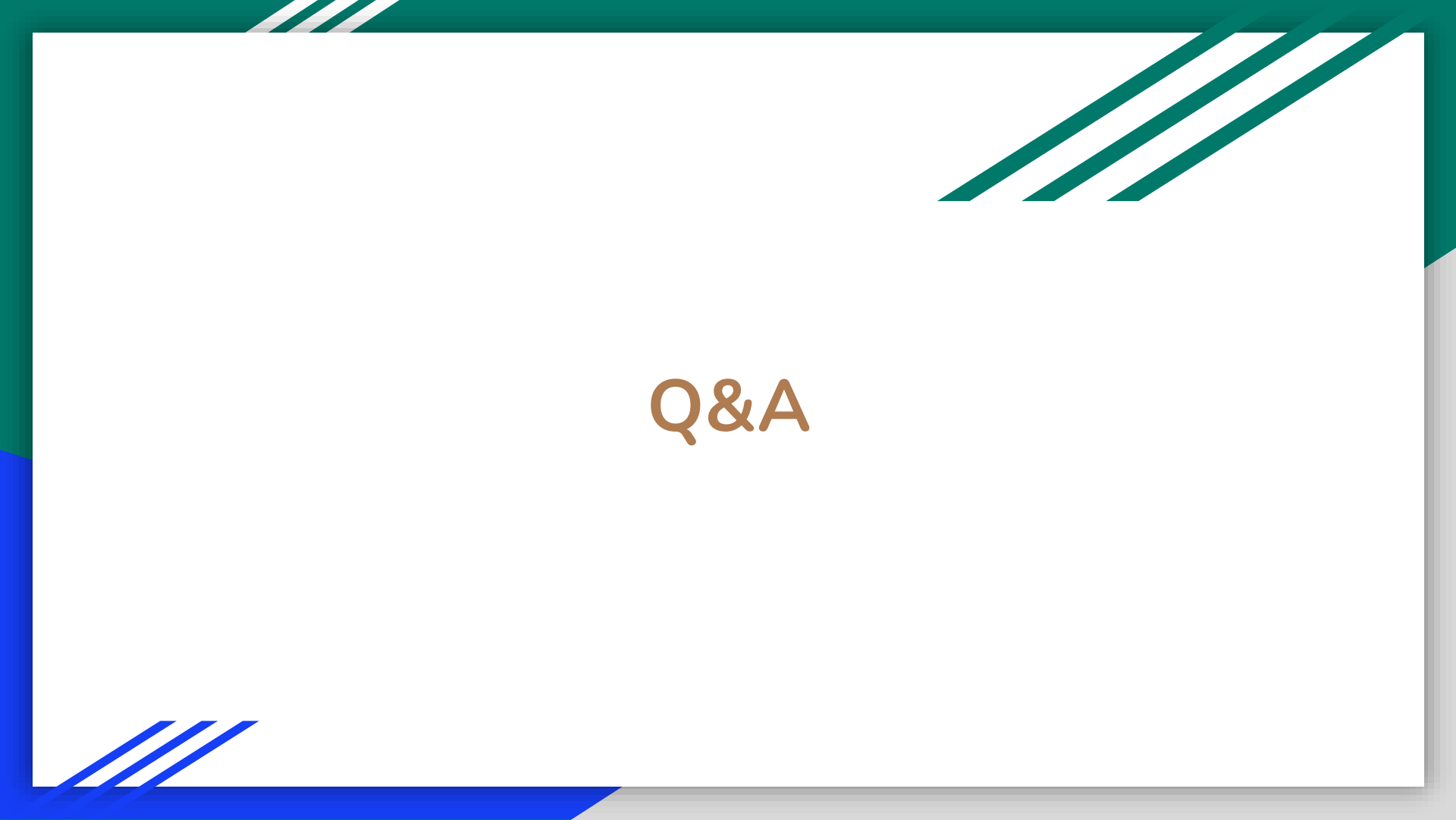
This is especially useful when you don't have a single production database, but every client has their own DB instance. Managing DB versions in such circumstances might become hell if you don't employ proper versioning techniques.

Another gain these best practices provide is a high cohesion of the database changes. It means that every notable modification in the schema and the reference data is reflected in a single place and not spread across the application.

3. What are the benefits of such approach?

The SQL upgrade scripts also grant high cohesion in a sense that they contain every DB change required for a feature, so it's easy to understand what modifications were made in the database in order to unlock a particular functionality. Keeping both schema and data changes related to each other in a single file also helps a lot.

The approach described in this post is applicable even if you didn't follow it from the very beginning. To put it into practice, you just need to create an initial script with the database schema you have right now in production and start changing it incrementally from that moment. The current version should become version #1 from which you can move further using the techniques we discussed above.



Q&A